



“操作系统”课程设计报告

Report on Project of Course “Operating System”

实验课程：“操作系统”课程设计 (Course Code 0601044)

Project of Course “Operating System”

主讲教师：邓劲柏

学 分 数：1 Credits

开设学院：计算机与软件学院 College of Computer & Software

开设时间：2025~2026 学年第一学期 1st Semester

成绩 Grade: _____

河 海 大 学

目录

1	前言	4
2	实验 1: 鲲鹏云 ECS 的构建及内核编译	4
2.1	实验目的	4
2.2	实验过程	4
2.2.1	鲲鹏云实验环境构建	4
2.2.2	内核编译	5
2.2.3	内核模块编程	7
2.3	总结	8
3	实验 2: 内存管理	9
3.1	实验目的	9
3.2	实验过程	9
3.2.1	KMALLOC 分配实验	9
3.2.2	VMALLOC 分配实验	10
3.2.3	KMALLOC 与 VMALLOC	10
3.2.4	思考题	11
3.3	总结	12
4	实验 3: 进程管理	12
4.1	实验目的	12
4.2	实验过程	12
4.2.1	创建内核进程	12
4.2.2	打印输出 CPU 负载	13
4.2.3	打印输出进程信息	14
4.2.4	使用 CGROUP 限制 CPU 核数	15
4.3	总结	15
5	实验 4: 中断和异常管理	16
5.1	实验目的	16
5.2	实验过程	16
5.2.1	使用 TASKLET 实现打印 HELLO WORLD	16
5.2.2	用工作队列实现周期打印 HELLO WORLD	17
5.2.3	信号捕获程序	18
5.3	总结	19
6	实验 5: 内核时间管理	19
6.1	实验目的	19
6.2	实验过程	19
6.2.1	打印当前时间	19

6.2.2	编写计数器	20
6.2.3	监控累加计算代码的运行时间	21
6.3	总结	21
7	实验 6: 设备管理	22
7.1	实验目的	22
7.2	实验过程	22
7.2.1	测试硬盘的写速率	22
7.2.2	测试硬盘的读速率	23
7.3	总结	23
8	实验 7: 文件系统	24
8.1	实验目的	24
8.2	实验过程	24
8.2.1	为 Ext4 文件系统添加扩展属性	24
8.2.2	注册自定义文件系统类型	25
8.2.3	在 /PROC 下创建目录	25
8.2.4	使用 STFS 文件系统传递内核模块参数	26
8.3	总结	27
9	实验 8: 网络管理	28
9.1	实验目的	28
9.2	实验过程	28
9.2.1	基于 SOCKET 的 UDP 发送接收程序	28
9.2.2	使用 TSHARK 抓包	29
9.2.3	使用 SETSOCKOPT 发送记录路由选项	30
9.3	总结	31
10	总结与体会	31

操作系统课程设计报告

1 前言

本次操作系统课程设计依托华为鲲鹏云 ECS 平台与国产 openEuler 操作系统，旨在构建面向 ARM64 异构架构的系统级开发视野。课程任务要求我们突破传统应用层编程的边界，下沉至 Linux 内核空间，通过一系列高强度的工程实践，系统性掌握内核编译构建、虚拟内存管理、进程调度策略、中断处理机制、文件系统抽象及网络协议栈等核心概念与关键技术。其核心目标在于通过编写内核模块与系统级代码，从底层视角剖析操作系统的资源管理逻辑与运行机理。

在具体的实施路径中，我完整执行了从基础环境搭建到高阶网络应用开发的八项核心任务。实践始于对内核源码的裁剪与编译，这一过程不仅完成了实验环境的初始化，更使我建立了对操作系统镜像生成与加载流程的宏观认知。随后，探索深入至内核的核心管控子系统：在内存管理环节，我通过对比 `kmalloc` 与 `vmalloc` 在物理与虚拟地址空间上的差异，验证了分页机制下的内存映射逻辑；在进程管理层面，利用 `cgroup` 技术对 CPU 资源进行精细化控制，复现了现代容器技术的资源隔离原理；针对操作系统关键的中断处理机制，则通过编写 `tasklet` 下半部程序，解决了原子上下文中的延时响应难题。

随着对内核理解的加深，实践范畴进一步扩展至文件系统与网络通信领域。在这一阶段，我不仅通过 VFS 接口开发了自定义文件系统与设备驱动，屏蔽了底层实现的复杂性，更深入剖析了网络协议栈的运作细节——从基础的 `Socket` 通信进阶至 IP 层协议选项的控制，成功实现了路由记录等高阶功能。

本次课程设计不仅是一次理论知识的验证，更是一次技术思维的重构。通过这八个层层递进的实验任务，我打通了从硬件驱动、内核空间到用户态应用的全栈技术链路，深刻理解了国产操作系统在支撑复杂计算任务时的架构设计与协同机制。

2 实验 1：鲲鹏云 ECS 的构建及内核编译

2.1 实验目的

1. 熟悉华为云鲲鹏 ECS 的购买、配置及 SSH 远程连接方法。
2. 掌握 openEuler 操作系统内核源码的获取、配置、编译及安装流程。
3. 掌握基本的 Linux 内核模块编写、编译、加载与卸载方法。

2.2 实验过程

2.2.1 鲲鹏云实验环境构建

在华为云控制台创建 VPC 和安全组，购买鲲鹏计算 ECS。

图 1 控制台界面

关键参数解析：

- 1) 基础网络配置 VPC：指定云资源所在的物理数据中心地理位置为华北-北京四，定义了私有网络的 IP 地址范围。
- 2) 安全组配置：允许外部网络通过 TCP 协议访问服务器的 22 号端口，22 端口是 SSH 服务的默认端口。允许互联网控制消息协议流量通过。

- 3) 弹性云服务器配置 ECS：选择基于 ARM 架构的华为鲲鹏处理器，而非传统的 x86 架构，学习国产化硬件平台下的操作系统底层原理。分配 4 个虚拟 CPU 核心和 8GB 内存，为私有云内的服务器分配一个公网 IP 地址。

使用 SSH 工具连接公网 IP。通过 ECS 控制台中的远程登录功能，连接弹性公网 IP，进入成功后，修改了主机名为 `h1` 便于区分识别。

图 2 登录成功界面

2.2.2 内核编译

2.2.2.1. 安装工具，构建开发环境

首先通过 yum 包管理器安装“DevelopmentTools”组件包，确保系统中具备 gcc、make 等基础编译工具。

图 3 工具安装成功

安装 bc、openssl-devel 等核心依赖包，用于处理内核数学运算及加密签名逻辑。这里必须安装 openssl-devel，因为现代 Linux 内核在编译时会对模块进行签名验证，缺少 SSL 库会导致编译中断。

使用 `--nogpgcheck` 参数关闭 GPG 签名检查，以确保在实验环境下软件包能够顺利安装，避免因证书过期或签名问题导致环境构建中断。

图 4 安装成功

图 5 openssl 安装成功

2.2.2.2. 系统备份以防更新失败

执行 `uname-a` 记录当前正在运行的内核版本为 4.19.90，作为实验前的初始状态对照。使用 `tar` 将 `/boot/` 目录下的引导文件打包为 `boot_origin.tgz`，并通过 `sz` 命令下载至本地保存。

此步骤是关键的风险防范措施，确保在后续新内核无法启动的情况下，可以通过备份文件恢复系统的引导配置。

图 6 lrzsz 安装成功

2.2.2.3. 内核源码下载解压

使用 `wget` 从 Gitee 代码仓下载 openEuler5.10.0 版本的内核源码压缩包。通过 `unzip` 命令对下载的 5.10.0-13.0.0.zip 进行解压，获取完整的 C 语言源代码目录。解压完成后进入 `kernel-5.10.0-13.0.0` 目录，该目录是后续所有编译操作的根路径。

图 7 内核源码安装成功

2.2.2.4. 编译内核

执行 `make mrproper` 彻底清理源码目录中可能存在的旧编译残留，保证编译环境的纯净。

通过 `make openeuler_defconfig` 加载 openEuler 针对鲲鹏 ARM 架构预设的默认配置文件，这是确保内核能与当前硬件兼容的关键。

执行 `make -j4 Image modules dtbs` 开始并行编译。其中 Image 是内核核心镜像，modules 是驱动模块，dtbs 是设备树文件。`-j4` 利用了鲲鹏服务器的多核性能来缩短编译时间。

图 8 配置成功

图 9 并行编译

图 10 内核编译成功

2.2.2.5. 安装内核

运行 `make modules_install` 将编译好的内核模块安装到系统的 `/lib/modules/5.10.0` 目录下。

执行 `make install` 将内核镜像、System.map 及配置文件安装至 `/boot` 目录，并自动触发引导程序的配置更新，使系统能够识别新版本内核。

图 11 内核模块安装成功

图 12 安装成功

2.2.2.6. 重启并监控系统状态

由于更换内核属于底层变更，必须执行 `reboot` 重启系统。此时通过华为云控制台提供的 VNC 远程登录方式进行实时监控。VNC 登录不同于 SSH，它允许用户在系统尚未启动完成时直接观察引导界面的变化，是在内核调试阶段不可或缺的监视工具。

图 13 vnc 登录成功

2.2.2.7. 登录并验证内核版本

系统启动时，在 GRUB 菜单中选择版本号为 5.10.0 的新内核条目进入系统。进入系统后执行 `uname-r`，输出结果显示由原本的 4.19.90 成功变更为 5.10.0。这标志着内核的获取、配置、编译及安装流程圆满完成，系统已成功运行在自行编译的内核环境之上。

图 14 新内核启动成功

2.2.3 内核模块编程

2.2.3.1. 准备源文件与 Makefile

首先通过 SSH 终端进入实验目录 `tasks_1-1/task3`，确认目录下已包含核心源码文件 `helloworld.c` 及编译规则文件 `Makefile`。

在实际操作中，可以使用 WinSCP 软件通过 SFTP 协议连接云服务器，将本地编写好的代码直接上传至指定目录。这种开发模式利用了本地编辑器的便捷性，同时确保代码在云端的鲲鹏 ARM 环境下运行。

图 15 进入实验一 task3 文件夹

2.2.3.2. 编译内核模块

在主目录中执行 `make` 命令，根据 `Makefile` 设定的规则调用内核构建系统。从编译日志可以看出，系统进入了 `kernel-5.10.0-13.0.0` 源码目录进行交叉编译，先后生成了对象文件 `.o`、模块信息文件 `.mod.o` 以及最终的内核模块二进制文件 `helloworld.ko`。

编译成功标志着代码语法正确，且与当前 5.10 内核版本的头文件兼容。

图 16 编译源文件成功

2.2.3.3. 加载内核模块并验证

使用 `insmod helloworld.ko` 指令将编译好的模块动态插入运行中的内核空间。

执行 `lsmod | grep helloworld` 进行验证，结果显示 `helloworld` 模块已存在于内存中，占用大小为 262144 字节，计数为 0，证明模块加载成功且运行正常。

图 17 加载模块

2.2.3.4. 卸载模块与内核日志分析

执行 `rmmod helloworld` 卸载模块，随后通过 `dmesg | tail -n5` 查看内核环形缓冲区的日志输出：

加载阶段：日志显示 `hello_init` 和 `Hello,world!`，对应源码中 `module_init` 指定的初始化函数，证明模块入口点执行成功。

卸载阶段：日志显示 `hello_exit`，对应 `module_exit` 指定的清理函数，证明模块资源已安全释放。

安全提示：出现的“`loading out-of-tree module taints kernel`”及签名验证失败警告是由于实验模块未经过 openEuler 官方签名，这在开发测试环境下属于正常现象，不影响功能逻辑。

图 18 卸载模块

2.2.3.5. 代码分析

内核模块不同于应用程序，没有 main 函数，而是通过 module_init 和 module_exit 宏注册初始化和清理函数。

下面进行代码剖析：

这是编写所有内核模块必装的头文件，包含了模块加载、卸载所需的函数原型、宏定义以及内核符号表的定义：

```
#include <linux/module.h>
```

该宏用于声明模块的许可证。设置为"GPL"通用公共许可证是为了确保模块能够调用内核中受版权保护的导出符号，并向系统声明该代码遵循开源规范：

```
MODULE_LICENSE("GPL")
```

__init 标记：该宏告诉内核，此函数仅在初始化期间使用。模块加载完成后，内核会释放该函数所占用的内存以节省空间。

printk 函数：这是内核空间的打印函数。由于内核不具备用户态的标准输出，其输出内容会被发送到内核环形缓冲区。

返回值：返回 0 表示模块初始化成功；若返回非 0 值，则 insmod 会提示加载失败。

```
int __init hello_init(void) {  
    printk("hello_init\n");  
    printk("Hello, world!\n");  
    return 0;  
}
```

__exit 标记：该宏表示该函数仅在模块卸载时调用。

```
void __exit hello_exit(void) {  
    printk("hello_exit\n");  
}
```

module_init(hello_init)：向内核注册模块的入口点，即告知内核在 insmod 时从哪个函数开始执行。

module_exit(hello_exit)：向内核注册模块的出口点，即告知内核在 rmmod 时执行哪个函数来清理现场。

module_init 并不是一个简单的函数调用。它是一个宏。它告诉 GCC 编译器，把 hello_init 这个函数的地址，专门写到编译出的二进制文件里一个叫 .init.text 的特殊区域去。

实验输出的 dmesg 日志中完整出现了 hello_init、Hello,world!以及 hello_exit 字符串，且顺序与 insmod 和 rmmod 的操作时机完全吻合。

2.3 总结

本次实验中，我基于鲲鹏云架构成功构建了 ECS 环境，并完成了 open Euler 操作系统内核的编译与安装。通过编写、编译并加载 helloworld 模块，我打通了从用户空间到内核空间的交互流程。

实验过程中，在加载模块时遇到了 "loading out-of-tree module taints kernel" 的警告。通过查阅日志和分析原理，我确认这是由于模块未包含在内核主线源码且未签名导致的正常安全机制，并未影响模块功能的正确运行。

此次实验让我深刻理解了 Linux 内核动态可加载模块的机制。特别是 `module_init` 和 `module_exit` 宏如何利用 GCC 属性将函数链接到特定的内核段，这种无需重启即可扩展内核功能的设计，充分体现了 Linux 内核的灵活性与高效性。

3 实验2：内存管理

3.1 实验目的

1. 掌握 `kmalloc` 和 `vmalloc` 的使用方法及区别。
2. 通过打印指针地址，分析 ARM64 架构下的内核内存布局。
3. 理解内核态内存分配与释放的配对原则。

3.2 实验过程

3.2.1 `kmalloc` 分配实验

3.2.1.1. 模块加载与卸载

在 `task1/task1` 目录下执行 `make` 指令，成功调用内核源码树完成编译，生成了目标内核模块 `kmalloc.ko`。随后使用 `insmod` 命令加载模块，通过 `dmesg` 查看内核环形缓冲区日志，观察到了初始化函数打印的 "Start kmalloc!" 标记及分配的内存地址。最后使用 `rmmmod` 卸载模块，日志显示 "Exit kmalloc!"，表明模块退出及内存释放流程执行正常。

图 19 编译模块

图 20 加载模块

日志打印出的两个内存指针地址分别为 `ffff000d3d9fc00` 和 `ffff0000c0226000`。根据该 Linux 内存布局，内核空间的虚拟地址起始于 `ffff000000000000`。实验中获取的地址均以 `fff` 开头且位于该范围内，有力地证明了 `kmalloc` 函数是在内核空间中分配内存。

图 21 卸载模块

3.2.1.2. 代码分析

头文件包含了内核中常用的内存分配函数（如 `kmalloc` 和 `kfree`）的定义。它是进行内核动态内存管理实验的必要前提：

```
#include <linux/slab.h>
```

`kmalloc` 是内核中用于分配连续物理内存的常用方法，下面代码分别请求了 1024 字节和 8192 字节的内存：

```
kmalloccmem1 = (unsigned char*)kmalloc(1024, GFP_KERNEL);
kmalloccmem2 = (unsigned char *)kmalloc(8192, GFP_KERNEL);
```

`GFP_KERNEL` 标志：表示该分配代表内核进程执行，可能会导致进程睡眠以等待空闲页面的释放。

在 `mem_module_exit` 函数中调用 `kfree()`。与用户态编程类似，内核态申请的内存必须手动释放，否则会导致内核内存泄漏，最终可能引发系统崩溃。

```
kfree(kmalloccmem1);
kfree(kmalloccmem2);
```

`kmalloc` 分配的内存不仅在虚拟地址上连续，在物理地址上通常也是连续的。

3.2.2 vmalloc 分配实验

3.2.2.1. 模块加载与卸载

通过编译并加载 `vmalloc.ko` 模块，成功调用 `vmalloc` 函数分配了三块不同大小的内存空间。从 `dmesg` 日志输出可见，返回的三个虚拟地址均以 `ffff8000` 前缀开头。这些地址精准落在了内核定义的 `vmalloc` 区域范围内，即起始地址为 `ffff800000000000`，这与前序实验中 `kmalloc` 使用的线性映射区地址形成鲜明对比。实验结果有力地验证了 ARM64 架构下 `vmalloc` 机制的内存布局特征，即在专用的高端内核虚拟地址空间中分配内存，以支持虚拟地址连续但物理地址可能不连续的大块内存需求。

图 22 模块编译

图 23 模块加载与删除

图 24

3.2.2.2. 代码分析

`<linux/vmalloc.h>` 头文件包含了 `vmalloc` 和 `vfree` 函数的声明。与 `kmalloc` 不同，`vmalloc` 用于申请物理上不一定连续、但虚拟地址空间上连续的大块内存：

```
#include <linux/vmalloc.h>
```

代码中分别尝试申请了 8KB、1MB 以及 64MB 的内存空间。

```
vmallocmem1 = (unsigned char*)vmalloc(8192);
vmallocmem2 = (unsigned char*)vmalloc(1048576);
vmallocmem3 = (unsigned char*)vmalloc(67108864);
```

`vmalloc` 专门用于分配大块内存。它通过修改页表，将不连续的物理页映射到连续的虚拟地址区间，因此其开销比 `kmalloc` 略大，且不能用于原子上下文。

在模块退出函数中，必须使用 `vfree` 逐一释放已申请的指针，以确保内核虚拟地址空间的回收：

```
vfree(vmallocmem1);
vfree(vmallocmem2);
vfree(vmallocmem3);
```

3.2.3 kmalloc vs. vmalloc

3.2.3.1. 虚拟地址空间的分布差异

kmalloc: 分配的地址通常位于内核空间的逻辑映射区。这一区域的虚拟地址与物理地址之间仅存在一个固定的偏移量，因此转换速度极快。

vmalloc: 分配的地址位于专门的 vmalloc 区域。该区域是专门为那些不需要物理连续但需要大块虚拟连续空间的请求而保留的。

3.2.3.2. 物理连续性与分配效率

kmalloc: 保证分配的内存存在物理上是连续的。这对于需要进行 DMA 操作的硬件设备驱动至关重要，但受内存碎片影响，申请超大块内存时容易失败。

vmalloc: 仅保证虚拟地址连续，物理上往往由多个不连续的页框拼凑而成。它通过修改系统页表来实现这种连续性，因此分配开销更大，且访问速度略慢于 kmalloc。

3.2.3.3. 适用场景与实验观察

kmalloc: 更适合分配频繁使用的小额内存。在实验中，分配 1KB 和 8KB 内存时响应迅速且稳定。

vmalloc: 更适合分配大容量缓冲区。本次实验成功分配了 64MB 的空间，这在 kmalloc 模式下很难保证成功，体现了其在处理大宗数据存储时的优势。

3.2.4 思考题

3.2.4.1. 内存泄漏、内存溢出、内存越界的定义

内存泄漏: 指程序中已动态分配的堆内存由于某种原因程序未释放或无法释放。这会导致系统内存的浪费，随着程序运行时间增长，可用内存越来越少，最终可能导致系统变慢或崩溃。

内存溢出: 指程序在申请内存时，没有足够的内存空间供其使用；或者更常见的情况是指缓冲区溢出，即向一个固定大小的内存缓冲区写入了超过其容量的数据，导致覆盖了相邻内存区域的合法数据。

内存越界: 指程序向系统申请了一块内存后，使用时超出了申请的范围。包括读越界和写越界。内存溢出通常是写越界的一种表现形式。

3.2.4.2. 分析程序

问题 1:

会产生内存泄漏。

函数 `function_which_allocates` 中使用 `malloc` 动态分配了 `45*sizeof(float)` 大小的堆内存。指针 `a` 是一个局部变量，存储在栈上。当函数执行完毕返回时，局部变量 `a` 被销毁，但它指向的堆内存并没有通过 `free()` 函数释放。由于指针丢失，程序再也无法访问或释放这块内存，从而导致内存泄漏。

问题 2:

会产生内存溢出和内存越界。

在 `main` 函数中，`longstring` 被填充了 255 个字符 'B'。在 `func` 函数中，目标数组 `buffer` 只有 16 个字节的空間。`strcpy(buffer, input)` 函数会将 `input` 指向的字符串完整复制到 `buffer` 中，直到遇到结束符 `\0`，而 `main` 函数中的循环甚至没有显式添加 `\0`，这可能导致 `strcpy` 读取更多内存，产生读越界。

程序试图将至少 255 个字节的数据写入只有 16 个字节的 buffer 中，这会导致数据写入超过 buffer 的边界，覆盖栈上相邻的内存数据。这属于典型的栈缓冲区溢出。程序运行通常会触发段错误。

3.3 总结

本实验，通过对比 kmalloc 与 vmalloc 的分配行为，成功验证了 ARM64 架构下内核空间的内存布局。实验观察到 kmalloc 分配的地址位于内核逻辑映射区，而 vmalloc 则位于专用的虚拟映射区，这与 openEuler 内核配置中 48 位虚拟地址空间的定义完全契合。

在代码实践中，我深刻理解了 kmalloc 基于 Slab 分配器实现物理连续分配的优势，其极高的分配效率使其成为驱动开发的首选。相比之下，vmalloc 虽然支持大跨度内存分配，但因涉及复杂的页表映射调整，存在较高的性能开销。

实验中对内存泄漏风险的分析强化了我的底层编程意识。在缺乏垃圾回收机制的内核态环境，严格遵循 kfree/vfree 与分配函数的对称配对原则，是防范系统崩溃、保障内核稳定运行的生命线。

4 实验3：进程管理

4.1 实验目的

1. 掌握内核线程的创建与销毁。
2. 学习读取系统负载及遍历系统进程。
3. 使用 cgroup 机制限制进程的 CPU 资源占用。

4.2 实验过程

4.2.1 创建内核进程

4.2.1.1. 模块加载与卸载

成功编译并加载 kthread.ko 模块，通过 dmesg 日志观察到 "Newkthreadisrunning" 循环输出，证明内核线程已在后台周期性运行。卸载模块时日志显示 "Killnewkthread"，表明线程正确响应了 kthread_stop 信号并安全退出。实验完整验证了内核线程的创建、周期性调度及终止的生命周期管理机制。

图 25 编译模块

图 26 加载并删除模块

4.2.1.2. 代码分析

该头文件包含了内核线程创建、运行及停止所需的所有函数声明和宏定义：

```
#include <linux/kthread.h>
```

在 Linux 内核中，无论是进程、线程还是内核线程，统一都用 task_struct 结构体来表示：

```
static struct task_struct *myThread = NULL;
```


`while(!kthread_should_stop())`是内核线程的标准写法。线程会进入一个循环，持续检查是否有外部指令要求其停止。如果没有停止信号，它将继续运行。`msleep(2000)`使线程进入睡眠状态 2000 毫秒，释放 CPU 资源，防止死循环导致系统假死。

```
static int print(void *data)
{
    while(!kthread_should_stop()){
        printk("New kthread is running.");
        msleep(2000);
    }
    return 0;
}
```

这是一个组合宏，它同时完成了 `kthread_create` 和 `wake_up_process` 两个动作。执行后，名为“new_kthread”的线程将立即开始执行 `print` 函数：

```
myThread = kthread_run(print, NULL, "new_kthread");
```

在模块卸载时调用。它会设置线程的停止标志位，并唤醒该线程，使其退出 `while` 循环并结束生命周期：

```
kthread_stop(myThread);
```

结果分析：

执行 `insmod kthread.ko` 后，通过 `dmesg` 可以观察到“Create kernel thread!”。随后，日志中每隔 2 秒会自动循环打印一次“New kthread is running.”。这证明内核线程已经脱离了 `insmod` 进程，在后台独立异步运行。

执行 `rmmod kthread` 时，观察到“Kill new kthread”。日志停止刷新，说明 `kthread_stop` 成功引导线程安全退出，未产生僵尸内核线程。

4.2.2 打印输出 CPU 负载

4.2.2.1. 创建内核进程

本实验在 `tasks_k/3/task2` 目录下成功编译并加载 `cpu_loadavg.ko` 模块。`dmesg` 日志输出“Thecpuloadavginoneminuteis:0.00”，表明模块成功在内核态读取到了系统当前的 1 分钟平均负载。卸载模块时显示的“Exitcpu_loadavg!”确认了模块功能的完整性与资源释放的正确性。

图 27 编译模块

图 28 加载并卸载模块

4.2.2.2. 代码分析

`<linux/fs.h>` 头文件包含了内核文件系统 VFS 的核心函数定义。它是实现内核态文件打开、读取、写入及关闭的基础。

```
filp_open("/proc/loadavg",O_RDONLY,0)
```

在内核态打开指定路径的文件。/proc/loadavg 是一个内核导出的虚拟文件，记录了系统 1、5、15 分钟的平均负载。使用 IS_ERR(fp_cpu) 检查返回的文件指针。内核中出错时不返回 NULL，而是编码在指针中的错误码，必须使用专用宏进行判定。

```
kernel_read(fp_cpu,buf_cpu,sizeof(buf_cpu),&pos)
```

内核态专用的读取函数。允许内核直接将数据从文件读取到内核缓冲区的变量 buf_cpu 中。pos 变量用于记录文件偏移量。读取完成后，内核会自动更新该指针的位置。

```
strncpy(tmp_cpu_load,buf_cpu,4)
```

由于负载数据以字符串形式存储，代码通过 strncpy 截取前 4 个字节，精确获取最近 1 分钟的负载数值。

```
filp_close(fp_cpu,NULL)
```

操作完成后必须关闭文件句柄，回收内核结构体资源，防止文件描述符泄露。

4.2.3 打印输出进程信息

4.2.3.1. 模块加载与卸载

本实验在 tasks_k3/task3 目录下成功编译并加载 process_info.ko 模块。从 dmesg 日志可见，模块加载后输出了 "Startprocess_info!" 以及当前处于运行状态的进程列表，包括内核工作线程 kworker/1:4 和正在执行加载命令的 insmod。日志中显示的 state:0 对应 Linux 内核中的 TASK_RUNNING 状态，有力地验证了模块成功遍历了系统进程链表并精准筛选出了当前处于运行状态的进程。最后卸载模块时输出 "Exitprocess_info!"，表明实验流程完整且正确。

图 29 编译模块

图 30 加载和卸载模块

4.2.3.2. 代码分析

<linux/sched/signal.h> 与 <linux/sched.h> 是内核中管理进程控制块 PCB 的核心头文件，包含了 task_struct 结构体定义及遍历宏：

```
#include <linux/sched/signal.h>
#include <linux/sched.h>
```

在 Linux 内核中，每一个进程都由一个 task_struct 实例表示。定义此指针用于在遍历过程中指向当前的进程描述符：

```
Struct task_struct *p;
```

`for_each_process` 是内核提供的一个宏，它本质上是一个 `list_for_each` 循环，利用双向链表结构遍历内核中的 `task_list`：

```
for_each_process(p){}
```

通过判断 `p->state` 是否为 0，筛选出当前正在执行或准备执行的进程，并打印其名称、进程号和状态：

```
if(p->state==0)
    printk("1)name:%s2)pid:%d3)state:%ld\n",p->comm,p->pid,p->state);
```

4.2.4 使用 `cgroup` 限制 CPU 核数

安装 `libcgroup` 用户空间工具，并挂载 `tmpfs` 到 `cgroup` 目录，随后挂载 `cpuset` 子系统以启用 CPU 集合管理功能。

在 `cpuset` 目录下创建子控制组 `mycpuset`，并通过写入配置文件的方式，限制该组仅能使用 0 号内存节点和 0-2 号 CPU 核心。

图 31 `cgroup` 下载成功

图 32 `mount` 挂载

图 33 挂载 `cpuset`

编译死循环测试程序 `while_long`，使用 `cgexec` 将其在 `mycpuset` 控制组中启动，通过 `top` 监控进程状态，并使用 `taskset` 工具读取进程的 CPU 亲和性信息进行验证。

图 34 打开新终端

图 35 测试 `cpu` 核数成功

`top` 命令显示目标进程 `while_long` 正在运行且 CPU 占用率极高，确认测试负载已正常加载。通过 `taskset-p` 查询该进程的 CPU 亲和性掩码，返回值为 7。在二进制表示中，7 对应 111，即第 0、1、2 位被置位，这代表进程被允许在 CPU0、CPU1 和 CPU2 上调度。通过 `taskset-pc` 查询，直接输出了 CPU 列表 0-2。

`taskset` 获取的掩码值与列表范围，与我们在 `mycpuset` 控制组中设定的 `cpuset.cpus` 参数完全一致。这证明 `cpuset` 子系统成功地将进程的执行范围约束在指定的物理核心集合内，实现了预期的硬件资源隔离目标。

4.3 总结

通过对内核线程创建与进程遍历的实践，我建立了对操作系统资源调度的宏观与微观认识。在 `cgroup` 资源限制实验中，我通过正确配置 `cpuset.mems` 和 `cpuset.cpus` 参数，成功将进程绑定在特定 CPU 核上运行；源码分析环节中，利

用 `for_each_process` 宏遍历 `task_struct` 让我直观看到了 PCB 中保存的进程状态信息，而 `cgroup` 的操作则让我深刻领悟了 Docker 等现代容器技术底层利用 `Namespace` 做隔离、利用 `Cgroup` 做资源限制的核心原理，理解了如何在内核层面实现对计算资源的精细化控制。

5 实验 4：中断和异常管理

5.1 实验目的

1. 掌握中断下半部机制：Tasklet 和 Workqueue 的区别与实现。
2. 掌握用户态下的信号捕获与处理。

5.2 实验过程

5.2.1 使用 tasklet 实现打印 helloworld

5.2.1.1. 模块加载与卸载

本实验在 `tasks_k/4/task1` 目录下成功编译并加载 `tasklet_interrupt.ko` 模块。dmesg 日志显示，在模块初始化打印 "Starttaskletmodule..." 后，紧接着输出了 "HelloWorld!taskletisworking..."。这一结果有力地证明了 tasklet 被成功调度并执行了预定义的处理函数。最后卸载模块时输出 "Exittaskletmodule..."，表明模块的加载、中断处理及卸载流程均运行正常。

图 36 编译模块

图 37 加载并卸载模块

5.2.1.2. 代码分析

`<linux/interrupt.h>` 是内核中断子系统的核心头文件，包含了 Tasklet、软中断 Softirq 以及工作队列 Workqueue 的接口定义：

```
#include <linux/interrupt.h>
```

`tasklet_struct` 是内核用于管理 Tasklet 任务的核心数据结构，包含了回调函数指针、状态位及传递的数据：

```
static struct tasklet_struct my_tasklet;
```

该函数即为中断执行的“下半部”逻辑。与硬中断不同，Tasklet 运行在软中断上下文中，允许稍微耗时的操作，且不会屏蔽硬件中断：

```
static void tasklet_handler(unsigned long data){...}
```

在模块初始化时，通过 `tasklet_init` 将定义的结构体、处理函数以及需要传递的参数进行绑定：

```
tasklet_init(&my_tasklet, tasklet_handler, 0);
```

调用 `tasklet_schedule` 并不是立即执行函数，而是将该 tasklet 标记为“挂起”状态，等待内核在合适的时机自动调用：


```
tasklet_schedule(&my_tasklet);
```

在卸载模块前调用 `tasklet_kill`，用于等待正在运行的 Tasklet 完成并防止新的调度，确保系统不会访问已释放的模块内存：

```
tasklet_kill(&my_tasklet);
```

5.2.2 用工作队列实现周期打印 helloworld

5.2.2.1. 模块加载与卸载

本实验在 `tasks_k/4/task2` 目录下成功编译并加载 `workqueue_test.ko` 模块。通过观察 `dmesg` 日志，可以清晰地验证工作队列的延时与周期性调度机制：

延时启动验证：

模块启动时间戳：3304.372968("Start...")

首次打印时间戳：3309.336204("HelloWorld!")

两者相差约 5 秒。这验证了代码中设定的初始延时 5*HZ 生效，工作队列成功被推后执行。

周期性执行验证：

第二次打印：3324.640224

第三次打印：3339.744211

每次打印的时间间隔约为 15 秒。这证明了在工作队列的处理函数中，任务在执行完后，又重新通过 `schedule_delayed_work` 将自己调度到了 15 秒后执行，从而实现了周期性任务。

实验结果完全符合设计预期，成功演示了 Linux 内核中工作队列处理延时任务及运行在进程上下文的特性。

图 38 编译模块

图 39 加载并卸载模块

5.2.2.2. 代码分析

`<linux/workqueue.h>` 提供了创建、调度及管理工作队列的核心接口，而 `<linux/delay.h>` 则用于提供实验所需的休眠函数。

```
#include <linux/workqueue.h>
#include <linux/delay.h>
```

`workqueue_struct` 用于定义自定义的工作队列指针，而 `delayed_work` 则是一个包含 `work_struct` 和定时器的复合结构，用于实现带有延时功能的任务。

```
static struct workqueue_struct *queue = NULL;
static struct delayed_work mywork;
```

该函数是工作队列被调度时执行的逻辑。与 Tasklet 不同，工作队列运行在进程上下文中，这意味着处理函数内部允许进行休眠或阻塞操作。

```
void work_handle(struct work_struct *work) { ... }
```

在初始化时，通过 `create_singlethread_workqueue` 在内核中创建一个专用的内核线程，用于顺序执行提交给该队列的所有任务。

```
queue = create_singlethread_workqueue("workqueue_test");
```

使用 `INIT_DELAYED_WORK` 宏将自定义的处理函数 `work_handle` 与 `mywork` 结构体进行绑定，准备后续调度。

```
INIT_DELAYED_WORK(&mywork, work_handle);
```

在循环中通过 `queue_delayed_work` 将任务提交至队列，并设置 `5*HZ` 后执行。随后的 `ssleep(15)` 使初始化进程休眠 15 秒，确保任务有足够时间被处理。

```
queue_delayed_work(queue, &mywork, 5 * HZ);  
ssleep(15);
```

在模块卸载时，首先调用 `flush_workqueue` 确保队列中所有待处理的任务已全部执行完毕，随后调用 `destroy_workqueue` 销毁队列并释放相关的内核线程及内存资源。

```
flush_workqueue(queue);  
destroy_workqueue(queue);
```

5.2.3 信号捕获程序

5.2.3.1. 捕获终端按键信号

本实验通过运行用户态程序 `catch_signal`，成功验证了 Linux 的信号处理机制。程序有效拦截了终端输入的 `Ctrl+C`、`Ctrl+\` 和 `Ctrl+Z` 信号，并未执行系统默认的终止或挂起操作，而是输出了自定义的提示信息。这一结果表明，用户进程可以通过注册信号处理函数，灵活地接管并自定义对各类异步中断信号的响应行为。

图 40 信号捕获

5.2.3.2. 代码分析

引入必要的信号处理与系统调用头文件，以支持信号定义、处理函数接口及 `getpid` 等标准函数的调用：

```
#include<signal.h>  
#include<unistd.h>
```

编写自定义的信号处理回调函数，当进程接收到注册信号时，内核会暂停当前执行流并跳转至此执行特定逻辑：

```
Void signal_handler(intsig){...}
```

在主程序启动时通过 `getpid()` 获取并打印当前进程的唯一标识符，便于用户在其他终端精准发送控制信号：

```
printf("Current process ID is %d\n",getpid());
```

调用 `signal()` 函数向内核登记映射关系，声明当进程接收到特定信号时应由指定的 `signal_handler` 接管处理：

```
signal(SIGINT, signal_handler);  
signal(SIGQUIT, signal_handler);  
signal(SIGTSTP, signal_handler);
```

通过 `for(;;)` 使程序进入无限循环的死循环状态，以确保进程持续驻留内存并异步等待信号的到达，

在处理完信号反馈后通过 `exit(0)` 主动终止进程，防止程序在信号捕获结束后返回死循环继续空转。

5.3 总结

本实验通过对比 `Tasklet` 与 `Workqueue`，深度厘清了内核异步执行的边界。我深刻理解到 `Tasklet` 作为软中断的延迟机制，运行于不可睡眠的原子上下文，必须满足极高的执行效率；而 `Workqueue` 运行于进程上下文，允许重新调度与睡眠，是处理非紧急耗时任务的最佳实践。

通过编写信号捕获程序，我直观掌握了信号作为“用户态软中断”的异步通信流程。实验中对 `SIGINT` 和 `SIGTSTP` 等按键信号的成功捕获，揭示了进程在内核态返回用户态前进行信号检测的关键时机，深入理解了信号机制如何模拟中断处理来完成进程间的异步通知。

掌握 `tasklet_schedule` 与 `schedule_work` 等 API 仅是实践基础，更重要的是领悟了操作系统通过“上半部快速响应、下半部延迟处理”来平衡硬件实时性与系统吞吐量这一核心矛盾的设计哲学。这种机制确保了内核在处理密集 I/O 时，既能快速释放硬件中断线，又能保证复杂逻辑的平稳执行。

6 实验 5：内核时间管理

6.1 实验目的

1. 掌握内核时间获取接口。
2. 使用内核定时器实现延时触发。
3. 测量代码执行耗时。

6.2 实验过程

6.2.1 打印当前时间

6.2.1.1. 模块加载与卸载

本次实验在 `tasks_5/task1` 目录下顺利完成了 `current_time.ko` 模块的编译与加载。从 `dmesg` 抓取的日志来看，模块初始化时成功调用了内核时钟接口，并准确打印出了系统当前的挂钟时间“2026-01-14 14:51:58”。这一结果说明我们在内核态下获取时间戳并将其转换为“年月日时分秒”格式的逻辑是完全正确的，最后卸载模块时的退出日志也证实了程序资源释放正常，整个实验流程符合预期。

图 41 编译模块

图 42 加载并卸载模块

6.2.1.2. 代码分析

引入内核时间处理、实时时钟转换及时间保持相关的头文件，为获取系统时间与格式化转换提供函数支持：

```
#include <linux/time.h>
#include <linux/rtc.h>
#include <linux/timekeeping.h>
```

定义 timespec64 结构体用于存储秒和纳秒级的原始时间戳，以及 rtc_time 结构体用于存储转换后的年、月、日等日期格式：

```
struct timespec64 ts;
struct rtc_time tm;
```

调用 ktime_get_real_ts64 函数获取自 Unix 纪元 1970 年 1 月 1 日以来的实时墙上时间，并将其保存至 ts 结构体中：

```
ktime_get_real_ts64(&ts);
```

利用 rtc_time64_to_tm 将获取到的秒数时间戳转换为人类可读的日期与时间分量，并填充到 tm 结构体中：

```
rtc_time64_to_tm(ts.tv_sec, &tm);
```

对转换后的数值进行逻辑修正，包括年份加 1900、月份加 1 以及针对北京时间进行的 8 小时偏移处理：

```
year = tm.tm_year + 1900;
mon = tm.tm_mon + 1;
hour = tm.tm_hour + 8;
```

6.2.2 编写计数器

6.2.2.1. 加载并编译模块

本次实验在 tasks_05/task_2 目录下完成了 timer_example.ko 模块的编译与加载。通过 dmesg-T 查看带时间戳的系统日志，可以清晰地观察到模块在 15:01:53 初始化启动，而定时任务触发打印 "hello,world!" 发生在 15:02:03。两者时间差精确为 10 秒，这与实验预期完全吻合，成功验证了 Linux 内核定时器的创建、激活及超时处理机制。

图 43 加载并编译模块

6.2.2.2. 代码分析

使用 `timer_setup` 宏初始化定时器，将自定义的 `print` 函数与定时器绑定，并设置其在软中断上下文中运行：

```
timer_setup(&timer, print, 0);
```

利用内核全局变量 `jiffies` 加上 $10 * \text{HZ}$ ，设定定时器在 10 秒后过期：

```
timer.expires = jiffies + 10 * HZ;
```

调用 `add_timer` 将配置好的定时器注册到内核的时间轮或高精度定时器队列中，使其正式生效开始计时：

```
add_timer(&timer);
```

在模块卸载时执行 `del_timer`，从内核激活队列中移除

6.2.3 监控累加计算代码的运行时间

6.2.3.1. 加载并编译模块

本实验在 `tasks_k5/task3` 目录下成功编译并加载 `sum_time.ko` 模块。`dmesg` 日志详细记录了计算任务的起始与结束时间戳，并验证了 1 至 100000 的累加结果为 5000050000，计算正确无误。通过对比起始与结束时间，日志最终输出该次累加运算在内核态的耗时为 720us。实验结果表明，利用内核时钟接口能够精确监控内核态代码的执行效率，且模块的加载、执行与卸载流程均符合预期。

图 44 加载并编译模块

6.2.3.2. 代码分析

定义一个简单的累加计算函数 `sum`，通过 `for` 循环执行十万次加法运算，模拟内核中需要进行耗时性能评估的计算任务：

```
static long sum(int num) { ... }
```

根据两个时间节点的差值，通过公式计算出计算任务执行的总耗时：

```
time_cost = (end - start) * 1000000 + end_u - start_u;
```

6.3 总结

在本次内核时间管理实验中，我通过直接操作 `timer_list` 结构体和核心变量 `jiffies`，深刻理解了 Linux 内核如何利用硬件时钟中断驱动时间的流逝。实验中我通过将 `jiffies` 与系统定义的 `HZ` 频率常量相结合，实现了精确的秒级延时与转换，这让我认识到内核态的时间度量本质上是系统节拍的累加。

在编写动态定时器代码时，我意识到内核延时机制与用户态有着本质区别：在不可睡眠的原子上下文中，必须依赖内核定时器通过非阻塞的异步回调来处理任务，而不能使用会导致进程挂起的忙等待，这种基于 `add_timer` 和 `mod_timer` 的事件调度机制，实际上是理解内核底层异步处理的核心。通过对 `proc_uptime` 数据读取与 `ktime_get_real_ts64` 接口的调用，我不仅掌握了如何获取系统自启动以来的精确时长，更领悟到这种时间管理机制是进程调度算法计

算时间片、以及驱动开发中处理 I/O 超时逻辑的底层基石，它确保了操作系统能够在毫秒级甚至微秒级精度下，维持多任务运行的有序性与实时性。

7 实验 6：设备管理

7.1 实验目的

1. 掌握在内核态读写文件的方法。
2. 编写模块测试硬盘读写速率。

7.2 实验过程

7.2.1 测试硬盘的写速率

7.2.1.1. 模块加载与卸载

本实验在 `tasks_k6/write_disk` 目录下完成了硬盘写速率测试模块的编译与运行。通过 `dmesg` 日志可见，模块加载后立即执行了写盘操作，耗时 `913593us`，测得写入速率高达 `587M/s`。这一结果表明内核态文件写操作功能正常，且得益于系统缓存或高性能存储介质，写入效率较高；最后卸载模块时的 `"Exit"` 日志确认了实验流程的完整闭环。

图 45 模块加载与卸载

7.2.1.2. 代码分析

通过宏定义设定单次写入的缓冲区大小为 1KB，并指定循环写入次数为 524288 次，旨在通过累计写入 512MB 数据的过程来压测内核的磁盘吞吐性能。

```
#define buf_size 1024
#define write_times 524288
```

利用循环逻辑为缓冲区 `buf` 填充初始数据，模拟真实的文件内容生成过程：

```
for(i = 0; i < buf_size; i++) { buf[i] = i + '0'; }
```

调用 `filp_open` 并在标志位中加入 `O_CREAT`，若指定路径 `/home/tmp_file` 的文件不存在则自动创建，并设置其访问权限为 `0644`：

```
fp_write = filp_open("/home/tmp_file", O_RDWR | O_CREAT, 0644);
```

通过 `kernel_write` 在循环中执行同步磁盘写入，内核会自动更新 `pos` 指针以维护文件内部的逻辑偏移位置：

```
kernel_write(fp_write, buf, buf_size, &pos);
```

根据写入总数据量与总耗时的比例关系，计算并输出磁盘的平均写入速率，直观量化内核态 I/O 的效率：

```
write_time = (write_end_time - write_start_time) * 1000000 /
(write_end_time_u - write_start_time_u);
```

```
printf("writing speed is %d M/s\n", buf_size * write_times / write_time);
```

7.2.2 测试硬盘的读速率

7.2.2.1. 加载与卸载模块

本实验在 `tasks_k/6/read_disk` 目录下顺利完成了硬盘读速率测试模块的编译与执行。根据 `dmesg` 日志反馈，模块加载后执行文件读取操作耗时 141315u，测得读取速率高达 3799M/s。这一速率表明读取的数据直接来自操作系统的页缓存，而非实际的物理磁盘 I/O；实验最后成功卸载了模块，验证了在内核态进行文件读取及性能测算的完整逻辑。

图 46 模块加载与卸载

7.2.2.2. 代码分析

设定宏定义 `read_times` 为 524288 次，配合 1KB 的缓冲区，旨在通过循环读取总量为 512MB 的数据来测试内核态的磁盘读性能：

```
#define read_times 524288
```

调用 `filp_open` 以只读模式打开前序实验创建的 `/home/tmp_file` 文件，建立内核态的文件访问上下文：

```
fp_read = filp_open("/home/tmp_file", O_RDONLY, 0);
```

使用 `kernel_read` 在循环中执行同步读取操作，该函数直接将数据从文件系统缓冲区拷贝至内核栈中的 `buf`，并由 `pos` 自动维护读取偏移量：

```
for(i=0; i < read_times; i++) { kernel_read(fp_read, buf, buf_size, &pos); }
```

数据读取完毕后再次捕捉时间戳并调用 `filp_close` 关闭文件句柄，确保内核资源被正确回收：

```
ktime_get_real_ts64(&ts);  
filp_close(fp_read, NULL);
```

通过计算读取总字节数与总耗时的比值，输出以 M/s 为单位的读取速率，直观反映 openEuler 内核在鲲鹏硬件上的磁盘读取效率：

```
read_time = (read_end_time - read_start_time) * 1000000 + (read_end_time_u -  
read_start_time_u);  
printf("Reading speed is %d M/s\n", buf_size * read_times / read_time);
```

7.3 总结

本实验打破了常规用户态编程的思维定式，通过直接调用 `filp_open`、`kernel_read` 和 `kernel_write` 等内核专用接口，实现了在内核空间对文件的直接访问。我深入理解了内核如何绕过用户空间指针检查，直接将数据在文件与内核缓冲区之间传输，这是驱动程序加载固件或读写配置文件的核心技术路径。

在对磁盘进行读写速率测试时，我观察到初次写入速度远高于物理磁盘理论上限。深刻认识到这是 Linux 页缓存机制在起作用。通过多次测试取平均值的方法，我消除了缓存带来的波动，获得了更接近硬件真实的性能数据。

这次实践让我透彻理解了 Linux“一切皆文件”的抽象哲学——即无论是真实文件还是设备节点，内核都通过 VFS 接口进行统一管理。同时，实验也警示我在内核编程中必须严谨处理 `filp_close`，严格遵循“谁打开谁关闭”的原则，以防止内核级的文件描述符泄漏导致系统资源耗尽。

8 实验 7：文件系统

8.1 实验目的

1. 操作 Ext4 扩展属性。
2. 理解 VFS 机制，注册自定义文件系统。
3. 利用 `/proc` 和 `sysfs` 与内核交互。

8.2 实验过程

8.2.1 为 Ext4 文件系统添加扩展属性

8.2.1.1. 环境检查与准备

实验首先通过 `dnf install` 确认了 `libattr` 工具包已经安装在系统中。接着使用 `df -Th` 检查文件系统，确认根目录/挂载点对应的设备 `/dev/vda2` 是 `ext4` 文件系统。

图 47 下载 libattr

图 48 执行 `fdisk` 与 `tune2fs` 进行检查

8.2.1.2. 扩展属性的设置与存储

实验成功演示了如何使用 `setfattr` 命令向文件写入不同编码格式的属性值。你分别设置了纯文本属性、包含转义字符的八进制值、十六进制数值以及 Base64 编码字符串。默认的 `getfattr` 输出显示，系统对于可打印的文本直接显示字符，而对于包含不可打印字符或二进制数据的属性，则自动采用 Base64 编码进行展示，证明了扩展属性机制能够灵活处理多种数据类型。

图 49 扩展属性的设置与存储

8.2.1.3. 多种编码格式读取验证

通过 `getfattr` 的 `-e` 参数验证了数据的底层一致性。使用 `-ehex` 时，所有属性均以十六进制显示，清晰地验证了 `user.hex` 的值确为 `0x0123`，且 `user.base64` 的值正确对应了 "HelloWorld!" 的十六进制表示。使用 `-ebase64` 时，所有属性统一转换为 Base64 字符串，如 `user.name` 变为 `0seGF0dHJfdGVzdA==`。这一系列对

照结果有力地证明了 Linux 文件系统扩展属性的存储是二进制安全的，而查看方式可以根据需求在文本、十六进制和 Base64 之间灵活切换。

图 50 多种编码格式测试

8.2.2 注册自定义文件系统类型

8.2.2.1. 注册自定义文件系统

查看系统中已经注册的文件系统类型：

图 51 查看已注册文件系统

本实验在 `tasks_k/7/task2` 目录下通过编译并加载 `register_newfs.ko` 模块，成功实现了自定义文件系统类型的注册。实验采用了加载前后对比的方式进行验证：

在模块加载前，查看 `/proc/filesystems` 未发现目标条目；

模块加载后，再次查询成功输出了 `nodevmyfs`，这里的 `nodev` 属性表明该文件系统属于伪文件系统，有力地证明了 `register_filesystem` 接口已被正确调用。

随后卸载模块，该条目随即消失，且 `dmesg` 日志完整记录了模块的 "Start" 与 "Exit" 过程，确认了自定义文件系统类型在内核中注册与注销流程的正确性。

图 52 注册自定义文件系统

8.2.2.2. 代码分析

定义 `file_system_type` 结构体用于描述新文件系统的元数据，其中 `.name` 指定了在系统中挂载时使用的名称标识 "myfs"：

```
static struct file_system_type myfs_type = { .name = "myfs", ... };
```

使用 `MODULE_ALIAS_FS` 宏为该文件系统创建一个内核别名，确保内核在尝试挂载名为 "myfs" 的文件系统时能正确关联到此模块：

```
MODULE_ALIAS_FS("myfs");
```

在初始化函数中调用 `register_filesystem` 向内核的 VFS 层注册该文件系统类型，使其正式成为内核支持的文件系统选项：

```
return register_filesystem(&myfs_type);
```

在模块卸载时执行 `unregister_filesystem`，从内核的全局文件系统链表中移除该类型，防止系统在卸载后继续尝试访问已失效的结构体：

```
unregister_filesystem(&myfs_type);
```

8.2.3 在 `/proc` 下创建目录

8.2.3.1 创建目录

本实验在 `task_k/7/task3` 目录下通过编写并编译 `proc_mkdir.ko` 模块，实现了在 Linux 伪文件系统 `/proc` 中创建自定义目录的功能。实验中，首先执行 `make` 完成编译，随后加载模块。加载后使用 `find/proc/-namemyproc` 命令进行验证，系统成功返回了 `/proc/myproc` 路径，证明目录已由内核模块成功创建。最后执行 `rmmod` 卸载模块，再次查找时该目录已不复存在，且 `dmesg` 日志完整输出了模块的 "Start" 与 "Exit" 信息，验证了该内核模块对 `/proc` 目录项的动态创建与销毁逻辑完全正确。

图 53 创建目录

8.2.3.2. 代码分析

定义一个 `proc_dir_entry` 结构体指针，该结构体是内核管理 `/proc` 目录下每一个条目的核心句柄：

```
static struct proc_dir_entry *myproc_dir;
```

调用 `proc_mkdir` 函数在 `/proc` 根目录下创建一个名为 "myproc" 的虚拟目录，若第二个参数为 `NULL` 则默认建立在 `/proc` 顶层：

```
myproc_dir = proc_mkdir("myproc", NULL);
```

通过对 `myproc_dir` 指针的空值判断来处理异常，若创建失败则返回 `ENOMEM` 错误码，告知系统内存不足或资源分配失败：

```
if(myproc_dir == NULL) return -ENOMEM;
```

在模块卸载时执行 `proc_remove` 函数，从虚拟文件系统中彻底移除该目录及其子条目，确保内核符号表和内存空间的整洁：

```
proc_remove(myproc_dir);
```

8.2.4 使用 sysfs 文件系统传递内核模块参数

8.2.4.1. 加载模块与卸载

本实验在 `tasks_k/7/task4` 目录下通过编译并加载 `sysfs_exam.ko` 模块，深入验证了 `sysfs` 文件系统作为用户空间与内核空间交互接口的功能。模块加载初期的 `dmesg` 日志显示参数初始化值为 `a=0,b=0,c="Hello,World"`。通过查看 `/sys/module/sysfs_exam/parameters/` 目录，发现仅存在 `b` 和 `c` 的对应文件而无 `a`，这精准地验证了代码中 `module_param` 宏权限设置的作用——即权限设为 0 的参数在 `sysfs` 中不可见，而其他参数则以文件形式暴露。

图 54 编译

图 55 加载模块

实验通过 `echo` 命令修改参数 `c` 的文件内容，并在卸载模块时的 `dmesg` 日志中观察到 `c` 的值变更为 "Happynewyear!"。这一过程有力地证明了用户可以通过

读写 sysfs 中的虚拟文件，实时、动态地修改运行中的内核模块变量，实现了内核行为的动态控制。

图 56 修改

图 57 卸载模块

8.2.4.2. 代码分析

引入内核属性文件系统权限相关的头文件，用于设置模块参数在 /sys/module 路径下的访问权限：

```
#include <linux/stat.h>
```

利用 module_param 宏定义内核模块参数，允许用户在加载模块时通过命令行动态传递数值：

```
module_param(a, int, 0);  
module_param(c, charp, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
```

通过 MODULE_PARM_DESC 为每个参数提供描述性字符串，这些信息可以通过 modinfo 命令查阅，增加了模块的可读性与易用性：

```
MODULE_PARM_DESC(b, "An visible int under sysfs");
```

设置参数 a 的权限为 0，这意味着该参数仅存在于内核内部，不会在 /sys 文件系统中创建对应的节点，用户无法通过文件查看或修改：

```
module_param(a, int, 0);
```

设置参数 b 和 c 的权限，使其在 /sys/module/[module_name]/parameters/ 目录下可见，且具备用户读写或组读取权限：

```
module_param(b, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
```

8.3 总结

通过对文件系统扩展属性、注册机制及内核交互接口的综合实践，我全方位掌握了 Linux 文件系统的层次结构。

实验中，我深入探究了 Ext4 文件系统的扩展属性 xattr 机制，利用 tune2fs 工具验证并确认了底层文件系统对 user_xattr 等特性的支持情况，并熟练使用 setfattr 为文件添加了多类元数据，深刻体会了文件系统通过 Key-Value 键值对关联元数据的灵活性。

在内核代码层面，我通过填充 file_system_type 结构体并调用 register_filesystem 函数，成功向内核 VFS 层注册了一个自定义的文件系统类型，直观理解了 VFS 如何通过统一的接口抽象来跟踪和管理不同的文件系统实现。

更进一步，我利用 proc_mkdir 在内存驻留的伪文件系统 proc 中创建了目录，并配合 module_param 宏将内核变量映射为 /sys 下的可见属性文件。

这些操作在用户空间与内核空间之间建立了直接的交互窗口，让我深刻认识到 Linux 文件的本质不仅是数据的物理容器，更是内核导出内部运行状态和控制设备驱动参数的通用接口，掌握这种通过虚拟文件系统进行内核交互的机制，对于进行复杂的系统管理和驱动参数调试具有极高的实用价值。

9 实验 8：网络管理

9.1 实验目的

1. 编写 SocketUDP 通信程序。
2. 使用 tshark 抓包分析。
3. 使用 setsockopt 设置 IP 选项。

9.2 实验过程

9.2.1 基于 socket 的 UDP 发送接收程序

9.2.1.1. 编译运行

本实验在 tasks_k8/task1 目录下完成了 UDP 网络通信程序的编译与验证。首先使用 gcc 指令顺利编译了 server.c 和 client.c 源文件，生成了对应的可执行程序。

图 58 gcc 编译

在运行阶段，客户端程序正确响应了用户输入的字符串并通过 Socket 发送；服务端程序则实时接收并准确打印了相同的内容。该结果表明客户端与服务端之间的 UDP 数据报传输链路畅通，验证了 sendto 和 recvfrom 等 Socket 接口在用户态进程间通信中的正确实现。

图 59 客户端

图 60 服务端

9.2.1.2. 代码分析

server.c:

引入 sys/socket.h 和 arpa/inet.h 等标准网络库，为建立 UDP 套接字及处理互联网地址协议提供函数支持：

```
#include <sys/socket.h>
#include <arpa/inet.h>
```

通过 socket 函数创建一个基于 IPv4 协议族和数据报套接字类型的文件描述符，用于非连接的 UDP 通信：

```
sock_fd = socket(AF_INET, SOCK_DGRAM, 0);
```

初始化 sockaddr_in 结构体，利用 htons 和 htonl 将主机字节序的端口号及 IP 地址转换为网络字节序：


```
server_addr.sin_port = htons(PORT);
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

执行 `bind` 操作将创建的套接字与指定的本地地址和端口绑定，使该程序开始在 40000 端口监听传入的数据包：

```
bind(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));
```

在 `while(1)` 循环中调用 `recvfrom` 函数阻塞等待接收数据，该函数能同时获取发送方的 `client_addr` 信息及数据负载内容：

```
recvfrom(sock_fd, buffer, BUF_SIZE, 0, (struct sockaddr*)&client_addr, &len);
```

通过 `printf` 实时输出接收到的缓冲区数据，展示了用户态网络程序如何通过系统调用接口与内核网络栈进行交互：

```
printf("The message received is: %s", buffer);
```

client.c:

利用 `inet_addr` 函数将点分十进制的本地回环地址 "127.0.0.1" 转换为网络字节的 32 位二进制 IP 地址，明确数据包的传输目的地：

```
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

通过调用 `read(0, buffer, BUF_SIZE)` 从标准输入获取用户输入的内容，实现了用户交互数据向程序的传递：

```
size = read(0, buffer, BUF_SIZE);
```

使用 `sendto` 函数将读取到的数据通过 UDP 套接字异步发送至预设的目标服务器地址，由于 UDP 是无连接协议，此处无需进行握手操作：

```
sendto(sock_fd, buffer, size, 0, (struct sockaddr*)&server_addr, len);
```

在 `while(1)` 循环结束后调用 `close(sock_fd)` 释放套接字资源，通知内核回收该网络链接占用的文件描述符和内存缓冲区：

```
close(sock_fd);
```

9.2.2 使用 tshark 抓包

在运行阶段，客户端程序正确读取并发送了用户输入的字符串，服务端程序实时接收并准确打印了相同内容，验证了基于 `sendto` 和 `recvfrom` 接口的 UDP 通信链路畅通。

进一步通过 `tshark` 工具监听本地回环接口，成功捕获到了源地址与目的地址均为 127.0.0.1、目标端口为 40000 的 UDP 数据包。底层抓包数据与应用层收发行为的一致性，完整证实了该 UDP 网络通信程序在 openEuler 系统下的正确实现与有效运行。

图 61 ifconfig 结果

图 62 客户端

图 63 服务端

图 64tshark 端

9.2.3 使用 setsockopt 发送记录路由选项

9.2.3.1. 发送记录路由选项

本实验在 tasks_k/8/task3 目录下完成了带有 IP 记录路由选项的 UDP 通信程序的编译与验证。首先使用 gcc 顺利编译了服务端与客户端源码。

图 65 编译

在运行阶段，客户端依次发送了"hi"、"how are you"和"i'm fine"三条测试消息，服务端均准确接收并打印，证明了在启用 Socket 选项后的 UDP 通信链路依然畅通。

与此同时，利用 tshark 工具在本地回环接口上实时捕获了端口 40000 的通信数据，并将抓包结果以 PDML 格式重定向输出至 setsockopt.xml 文件中。

图 66 客户端

图 67 服务端

图 68 tshark 端

9.2.3.2. 代码分析

client.c:

定义一个 28 字节的缓冲区 rrbuff，并按照 IP 协议首部选项字段的格式手动填充。其中 0x07 代表记录路由选项类型，随后设置长度及偏移量指针，用于要求路径上的路由器记录所经接口的 IP 地址：

```
rrbuff[0] = 0x07;
rrbuff[1] = sizeof(rrbuff) - 1;
rrbuff[2] = 4;
```

利用 setsockopt 函数在 IPPROTO_IP 级别设置 IP_OPTIONS。这一操作允许用户态程序干预内核网络栈的底层发包逻辑，将自定义的路由选项插入到每一个发出的 IP 数据报首部中：

```
ret = setsockopt(sock_fd, IPPROTO_IP, IP_OPTIONS, (void*)rrbuff,
sizeof(rrbuff));
```

通过检查 ret 返回值来确认内核是否允许该套接字携带自定义选项。在某些加固的内核环境中，修改 IP 选项可能需要特定的用户权限或受到内核安全策略的限制：

```
if(-1 == ret){ printf("setsockopt error!\n"); ... }
```

在 while(1) 循环中，每次调用 sendto 发送用户输入内容时，内核都会自动将之前通过 setsockopt 设定的记录路由选项附加在 IP 包头中，实现了应用层数据与网络层控制信息的同步传输：

```
sendto(sock_fd, buffer, size, 0, (struct sockaddr*)&server_addr, len);
```

server.c:

定义 sockaddr_in 结构体用于存储服务器自身和远程客户端的地址信息，这是处理 IPv4 网络地址的基础数据结构：

```
struct sockaddr_in server_addr, client_addr;
```

通过 htonl(INADDR_ANY) 将套接字绑定到本机的任意可用网卡地址，使得服务端能够接收来自系统所有网络接口的 UDP 数据包：

```
server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
```

调用 bind 将之前定义的 IP 和端口号与套接字关联，正式在内核中注册该监听地址，确保发往该端口的数据能准确投递给本程序：

```
if(-1 == bind(sock_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)))
```

使用 bzero 函数在每次循环接收前清空缓冲区，防止旧数据残留影响新接收消息的显示准确性：

```
bzero(buffer, BUF_SIZE);
```

核心函数 recvfrom 在接收数据负载的同时，会自动获取发端方的地址信息并填充到 client_addr 中，这为后续可能的双向通信提供了目标凭证：

```
if(-1 != (recvfrom(sock_fd, buffer, BUF_SIZE, 0, (struct  
sockaddr*)&client_addr, &len)))
```

9.3 总结

本实验贯通了从应用层 SocketAPI 到底层网络协议栈的完整链路。在构建基于无连接的 UDP 通信架构中，我深入掌握了云环境下的网络安全配置，认识到安全组作为虚拟防火墙在流量控制中的关键作用，并建立起“网络连通性优先”的系统化编程思维。在此基础上，我利用 setsockopt 函数针对 IPPROTO_IP 协议层进行精细化控制，通过开启记录路由选项，亲身体验了如何跨越网络层级直接操纵 IP 协议头部，验证了带内信令控制的强大能力。进一步结合 tshark 命令行工具在 Loopback 接口的实时抓包分析，我直观看到了数据包在传输层与网络层间的逐层封装过程，深刻理解了 Socket 本质上是作为应用层与 TCP/IP 协议族之间的抽象门面，通过屏蔽底层复杂性并协同各层协议，最终实现了可靠或高效的数据传输目标。

10 总结与体会

本次操作系统课程设计是一次极具挑战性也极具收获的深度学习，它成功地将抽象的操作系统理论概念转化为具象的代码逻辑与运行现象。

通过在华为云鲲鹏 ECS 上的一系列实验，我深刻体会到了从用户态编程向内核态编程思维转变的必要性与艰难性。在内核的世界里，我必须时刻保持对系统资源的敬畏，无论是手动管理内存的分配与释放，还是在中断上下文中严谨地处理并发与同步，每一个微小的逻辑漏洞都可能导致整个系统的崩溃，这种严谨的编程体验极大地锻炼了我的代码规范性与逻辑思维能力。

在技术层面，我不仅掌握了内核模块的开发流程，更深入理解了 Linux 内核的设计哲学。从内存管理实验中对物理地址与虚拟地址映射的直观认识，到进程管理中通过遍历进程控制块理解调度机制，再到文件系统实验中通过注册自定义文件系统领悟一切皆文件的抽象之美，这些实践让我看到了操作系统作为软硬件桥梁的复杂与精妙。

同时，在解决诸如内核编译依赖缺失、模块加载签名警告、并发编译优化以及网络通信故障等实际问题的过程中，我熟练掌握了内核日志调试、源码分析、自动化构建以及网络抓包等系统级开发与运维工具，极大地提升了我的工程实践能力与问题排查效率。

此外，本次实验基于国产 openEuler 操作系统与鲲鹏架构，让我切身感受到了国产基础软件生态的蓬勃发展与技术底蕴。这不仅增强了我的专业自信，也让我认识到掌握底层核心技术对于构建安全可控数字基础设施的重要性。总而言之，这段经历不仅加深了我对操作系统核心原理的理解，更培养了我面对复杂系统问题的分析能力与解决信心，为未来的底层系统软件开发与研究奠定了坚实的基础。